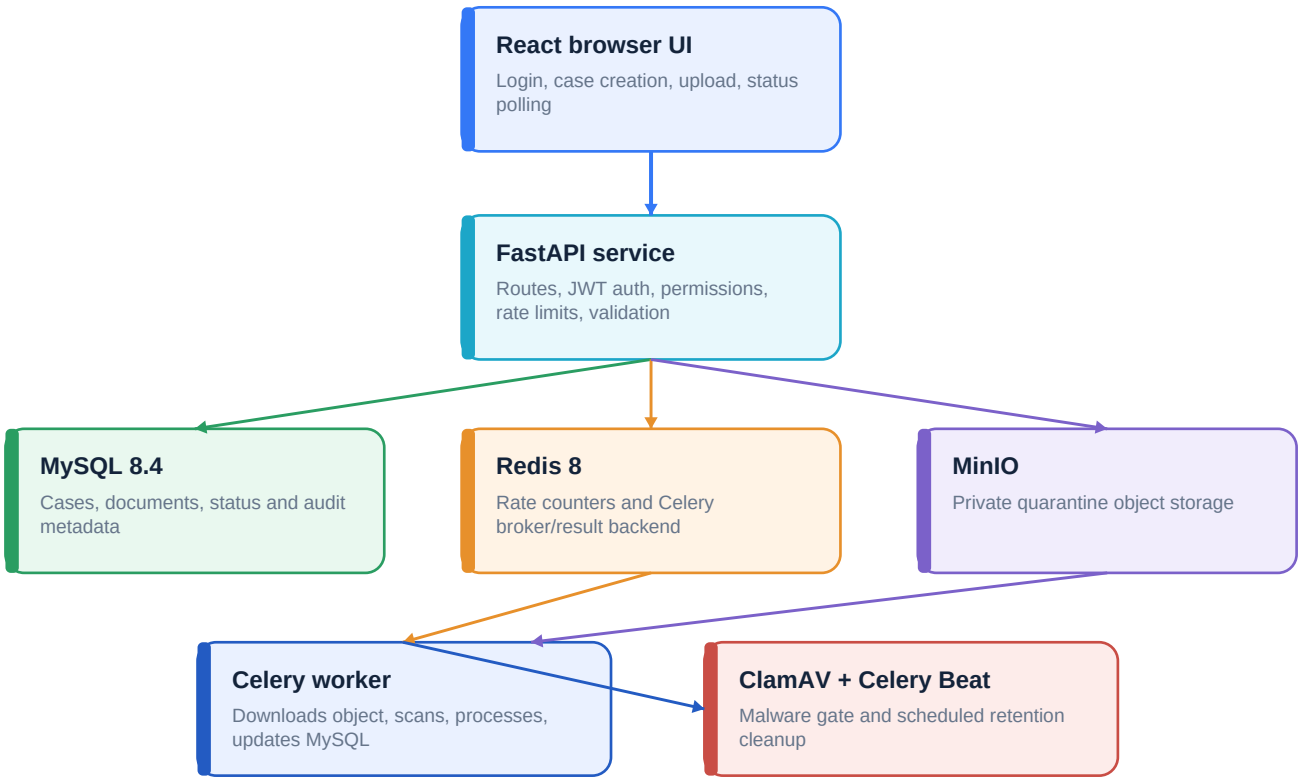
GRD Document Verification Technical Issue Analysis and Resolution Handbook

A consolidated analysis of the setup, runtime, upload, database, queue, storage, frontend and developer-tool issues encountered during implementation. Each section records the symptom, root cause, applied resolution, example scenario and useful command sequence.



Security note: commands use placeholders. Real JWT secrets, database passwords and account data are intentionally not reproduced.
Version 1.0 | 08 August 2026

1. Issue register

The register below provides a quick index. Detailed explanations and commands follow on later pages.

ID	Issue	Primary cause	Resolution status
I-01	No module named pydantic	Wrong Python interpreter / individual file execution	Resolved with project virtual environment and module launch
I-02	Virtual environment would not activate	Used cd on a PowerShell script	Resolved with the call operator
I-03	VS Code F5 launched auth.py	Incorrect debug target and Python 3.14	Resolved with launch.json for Uvicorn and the backend venv
I-04	Uvicorn IndentationError	Unexpected indentation in documents.py	Source corrected and compile/test checks added
I-05	Redis/Celery purpose unclear	File storage and task messaging were confused	Architecture separated file bytes from queue messages
I-06	MySQL port not published	Existing container had no host mapping	Container recreated with 127.0.0.1:3306:3306 and named volume
I-07	Where to put database password	System password confused with DB credential	DATABASE_URL documented and credentials aligned
I-08	Submission failed	Backend/database/queue dependencies not consistently available	Health-first diagnosis and error detail handling
I-09	Password-protected PDF rejected	Policy loaded as reject or process not restarted	Policy changed to review and services restarted
I-10	Document observations not visible	UI showed cases without processing detail	Database-backed observations displayed inside case documents
I-11	Repeated GET requests and changing ports	Frontend polling and normal client ephemeral ports	Explained and retained as expected behavior
I-12	Dashboard changes in wrong directory	Edits targeted a different frontend	Work restricted to grd-document-verf/frontend/src

1. Issue register - continued

ID	Issue	Primary cause	Resolution status
I-13	Base dashboard must remain	Feature page risked replacing existing UI	Verification functions kept under More / command centre
I-14	ClamAV container communication	Code expected local clamscan executable	ClamAV installed inside Celery worker image
I-15	No rate limits	Expensive endpoints were unprotected	Redis-based user/tenant/login/upload limits added
I-16	No processing retry	Failed work required re-upload	Tenant-scoped retry endpoint and UI action added
I-17	Local quarantine not production-ready	API and worker required shared local disk	Local/MinIO storage adapter introduced
I-18	No automatic expiry	Quarantine content could remain indefinitely	Celery Beat retention cleanup added
I-19	Statuses were too technical	Legacy OCR/extraction status names	Clear public lifecycle stages added
I-20	Git bad revision refs/heads/main	Repository had no first commit/main history	Initial commit and branch setup documented
I-21	Open authentication link in Firefox	VS Code external browser not configured	Workspace browser preference set to Firefox
I-22	Access MySQL data/procedures in container	Container filesystem confused with database persistence	docker exec, SQL views/procedures, volumes and backups documented
I-23	Unclear file-analysis libraries	Python logic and external tools were conflated	PyMuPDF, Pillow and ClamAV responsibilities documented
I-24	Multi-service startup was manual	MySQL, Redis, worker and API started separately	Docker Compose orchestration added

Outcome: automated backend tests, Python checks, frontend build, Compose validation and rendered documentation checks now provide repeatable evidence instead of relying only on manual observation.

2. Python environment and VS Code execution

I-01: ModuleNotFoundError for pydantic

Observed symptom	Running a schema file directly produced: No module named 'pydantic'.
Root cause	VS Code used a system Python installation instead of backend/.venv. Running app files directly also bypassed the application package startup path.
Resolution	Select backend/.venv/Scripts/python.exe and launch Uvicorn as a module. Install requirements only inside that environment.
Validation	python -c imports pydantic; Uvicorn imports app.main successfully.

```
cd D:\newdata\grd_document_verf\grd-document-verf\backend
& .\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -c "import pydantic; print(pydantic.__version__)"
python -m uvicorn app.main:app --host 127.0.0.1 --port 8003 --reload
```

I-02: activation command used with cd

PowerShell **cd** changes directories; it does not execute scripts. From the backend directory, run the activation script with **&** or dot-source it. If already inside .venv, do not add another .venv path segment.

```
# Correct from backend
& .\.venv\Scripts\Activate.ps1
# Correct while already inside backend\.venv
& .\Scripts\Activate.ps1
```

I-03: F5 launched auth.py instead of the application

The working launch configuration uses module=uvicorn, cwd=backend, envFile=backend/.env and the project virtual-environment interpreter. Individual endpoint modules are imported by FastAPI; they are not standalone programs.

3. API startup and Python source errors

I-04: IndentationError in documents.py

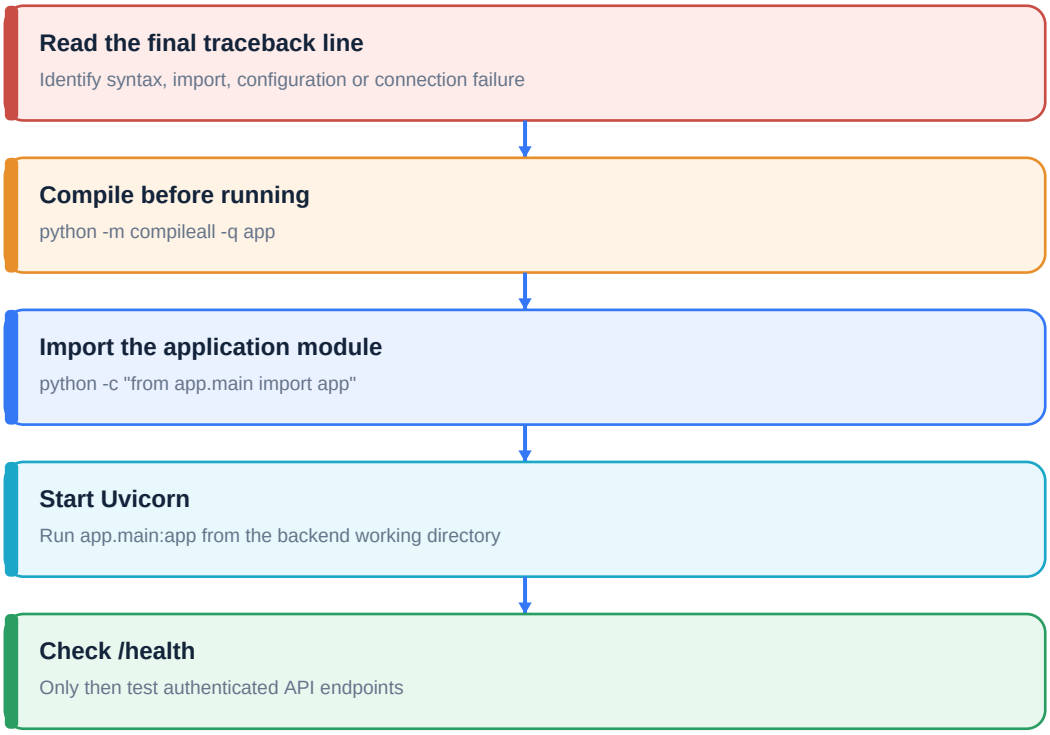
Observed symptom	Uvicorn reloader started, but the child process stopped at documents.py line 129 with unexpected indent.
Root cause	Python parses every imported endpoint before the server can accept traffic. One extra indentation level made the module invalid.
Resolution	Correct the function indentation, compile the complete app tree, then restart the Uvicorn module instead of the individual file.
Validation	compileall passes and /health returns status ok.

```
cd D:\newdata\grd_document_verf\grd-document-verf\backend
& .\.venv\Scripts\Activate.ps1
python -m compileall -q app
python -m uvicorn app.main:app --host 127.0.0.1 --port 8003 --reload
curl.exe http://127.0.0.1:8003/health
```

Example scenario

A developer presses F5 and sees 'Uvicorn running' followed by a traceback. The first line is only the reloader process. The final traceback line is decisive: IndentationError at documents.py. Fix that source error, save the file and let the reloader import app.main again.

Recommended diagnostic order



4. Correct local startup order

The local workflow uses separate terminals so that the API and worker remain visible. Infrastructure must be healthy before work is submitted.

Order	Component	Command / result
1	Docker Desktop	Docker engine must be running.
2	MySQL	docker start grd-mysql; port 3306 must be published.
3	Redis	docker start grd-redis; redis-cli ping returns PONG.
4	Database schema	Backend startup creates tables and applies additive compatibility updates.
5	FastAPI	Uvicorn listens on fixed port 8003.
6	Celery worker	Worker connects to redis://localhost:6379/0 and reports ready.
7	Frontend	Vite listens on 5176 and proxies /api to 8003.

Infrastructure commands

```
docker start grd-mysql
docker start grd-redis
docker exec grd-redis redis-cli ping
docker port grd-mysql
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

API terminal

```
cd D:\newdata\grd_document_verf\grd-document-verf\backend
& .\.venv\Scripts\Activate.ps1
python -m uvicorn app.main:app --host 127.0.0.1 --port 8003 --reload
```

Worker terminal

```
cd D:\newdata\grd_document_verf\grd-document-verf\backend
& .\.venv\Scripts\Activate.ps1
python -m celery -A app.workers.celery_app.celery_app worker --loglevel=info --pool=solo
```

Frontend terminal

```
cd D:\newdata\grd_document_verf\grd-document-verf\frontend
npm run dev
```

5. MySQL container, credentials and persistence

I-06: no public port 3306

Observed symptom	docker port grd-mysql 3306 reported that no public port was published.
Root cause	Docker port mappings are fixed when a container is created. docker start cannot add a missing mapping.
Resolution	Recreate the container with 127.0.0.1:3306:3306 and mount the named grd-mysql-data volume.
Validation	docker port shows 127.0.0.1:3306 and the API connects using localhost:3306.

```
docker run -d --name grd-mysql -p 127.0.0.1:3306:3306 `
-e MYSQL_DATABASE=grd_document_verf `
-e MYSQL_USER=grd `
-e MYSQL_PASSWORD=<DB_PASSWORD> `
-e MYSQL_ROOT_PASSWORD=<ROOT_PASSWORD> `
-v grd-mysql-data:/var/lib/mysql mysql:8.4
```

I-07: which password belongs in .env

DATABASE_URL uses the MySQL account password, not the Windows password. Its format is **mysql+pymysql://user:password@host:port/database**. URL-encode special characters. Never commit the active .env file.

```
DATABASE_URL=mysql+pymysql://grd:<DB_PASSWORD>@localhost:3306/grd_document_verf
REDIS_URL=redis://localhost:6379/0
CELERY_DISPATCH_ENABLED=true
```

Initialization warning

MySQL environment variables are applied only when /var/lib/mysql is initialized. Reusing an existing volume preserves old users/passwords. Changing Compose variables later does not rewrite them; use ALTER USER or intentionally initialize a new volume after a verified backup.

6. MySQL data, views, procedures and backups

I-22: database exists inside a container

The MySQL server process runs in the container, but durable table data lives in the named Docker volume. SQL objects such as views and stored procedures live inside the database schema and are created with normal SQL commands.

Open MySQL and inspect application data

```
docker exec -it grd-mysql mysql -ugrd -p grd_document_verf
SHOW TABLES;
SELECT id, filename, status, malware_scan_status FROM documents ORDER BY created_at DESC;
SHOW CREATE TABLE documents\G
```

Example view

```
CREATE OR REPLACE VIEW vw_document_processing_status AS
SELECT tenant_id, case_id, id AS document_id, filename, status,
malware_scan_status, storage_deleted_at, created_at
FROM documents;
SELECT * FROM vw_document_processing_status;
```

Example stored procedure

```
DELIMITER //
CREATE PROCEDURE GetCaseDocuments(IN p_case_id CHAR(36))
BEGIN
SELECT id, filename, status, malware_scan_status
FROM documents WHERE case_id = p_case_id ORDER BY created_at;
END //
DELIMITER ;
CALL GetCaseDocuments('<CASE_UUID>');
```

Backup and restore

```
New-Item -ItemType Directory -Force .\backups
docker exec grd-mysql sh -c `
'exec mysqldump -ugrd -p"$MYSQL_PASSWORD" grd_document_verf' `
> .\backups\grd.sql
Get-Content .\backups\grd.sql | docker exec -i grd-mysql sh -c `
'exec mysql -ugrd -p"$MYSQL_PASSWORD" grd_document_verf'
```

Production practice: use managed backups or scheduled dumps, monitor restore tests, retain database and MinIO backups separately, and never treat the container writable layer as durable storage.

7. Upload failure diagnosis

I-08: browser displayed Submission failed

Observed symptom	The upload form accepted a file but displayed a generic submission failure.
Root cause	A frontend message can represent API unavailability, authentication failure, database connection error, validation rejection, duplicate content, Redis queue failure or quarantine storage failure.
Resolution	Return API detail to the UI, then diagnose dependencies in order: health, logs, authentication, database, Redis, MinIO/local storage and file validation.
Validation	A valid file returns 202 Accepted; cases/status endpoints return 200; the worker advances the stored status.

Diagnostic commands

```
curl.exe http://127.0.0.1:8003/health
docker compose ps
docker compose logs --tail=200 backend mysql redis minio celery
docker exec grd-redis redis-cli ping
docker exec -it grd-mysql mysql -ugrd -p -e "SELECT 1" grd_document_verf
```

Example scenario: queue unavailable

The API successfully validates and stores statement.pdf, commits the document row, then cannot publish the Celery message because Redis is stopped. The secure upload transaction removes the newly created record and quarantine object and returns 503. After Redis returns PONG, resubmit the document.

Example scenario: duplicate file

The same tenant uploads identical bytes under a different filename. The SHA-256 value matches the existing row, so the new quarantine object is deleted and the API returns 409 with the existing document ID. Renaming a file does not bypass duplicate detection.

Status codes are useful evidence: 401 token problem, 403 permission problem, 409 duplicate/state conflict, 413 size/page limit, 415 file type mismatch, 422 corrupt/invalid document, 429 rate limit and 503 dependency unavailable.

8. Password protection, observations and analysis libraries

I-09: password-protected PDFs were still rejected

Observed symptom	Bank statements continued to show Password-protected PDFs are not accepted after the setting was changed.
Root cause	The active process had already loaded settings, the wrong .env was edited, or the policy value remained reject.
Resolution	Set PASSWORD_PROTECTED_PDF_POLICY=review in backend/.env and restart both API and worker. Review does not decrypt the file; it stores and scans the encrypted PDF and routes it to MANUAL_REVIEW.
Validation	A protected PDF returns 202 and eventually shows MANUAL_REVIEW rather than an intake rejection.

```
PASSWORD_PROTECTED_PDF_POLICY=review
docker compose up -d --force-recreate backend celery
docker compose logs -f backend celery
```

I-10: observations were not shown

Document responses now build observations from the database record: detected type, size, page count, password state, malware outcome, processing status and storage expiry. The case dashboard expands each document and displays those observations.

I-23: what actually checks the document

Component	Checks performed
Pure Python logic	Extension allow-list, chunked size limit, random filename, SHA-256 and duplicate lookup.
Magic signature logic	PDF, JPEG, PNG and TIFF header bytes; browser MIME is ignored.
PyMuPDF (fitz)	PDF open/integrity, pages and password requirement.
Pillow	JPEG/PNG/TIFF decoding, encoded format and frame/page count.
ClamAV	Malware scan before parser/OCR/verification processing.
SQLAlchemy/MySQL	Unique tenant hash, statuses and durable audit metadata.

Therefore the scan is not 'Python without libraries'. Python orchestrates the controls, while specialized libraries and the ClamAV executable perform file-format and malware analysis.

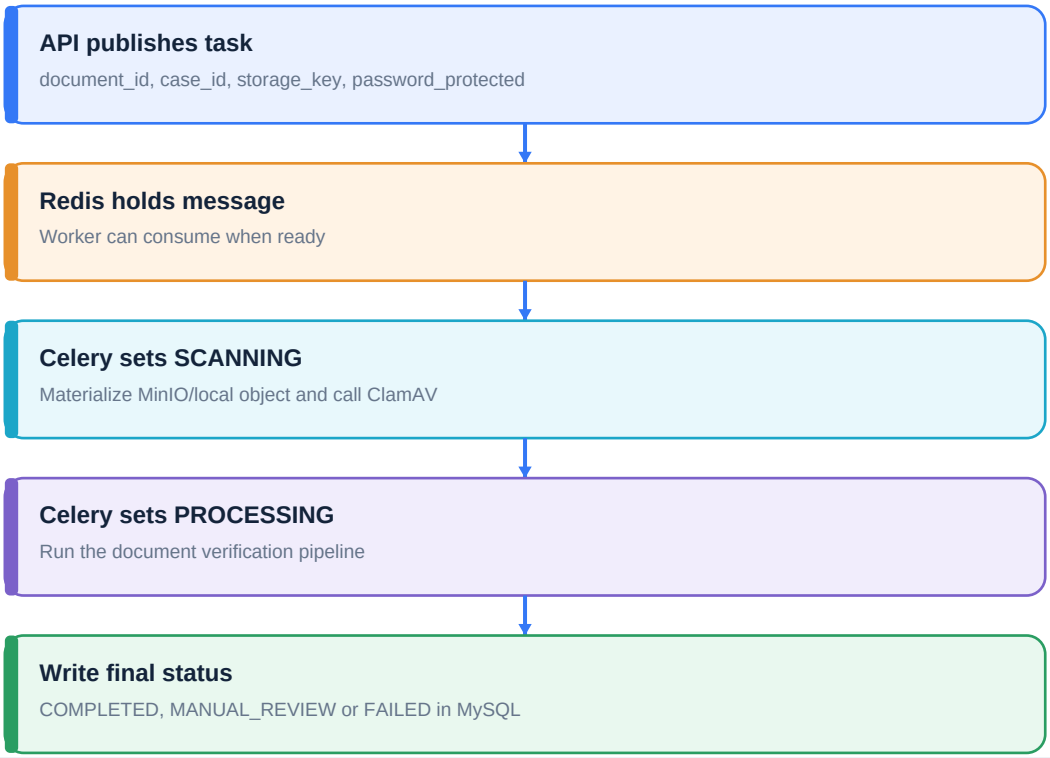
9. Redis, Celery and the background job

I-05: where Redis is called to upload the CV

Observed symptom	No code appeared to upload the CV/PDF into Redis.
Root cause	Redis is intentionally not file storage. It stores small Celery task messages and rate-limit counters. File bytes belong in MinIO or local quarantine.
Resolution	After storage and database commit, enqueue_document_processing calls verify_document.delay with IDs, storage_key and password flag. Celery publishes that message to Redis.
Validation	Worker logs show the received verify_document task and MySQL status changes from SCANNING to PROCESSING to a final state.

System	What it stores	Retention
MySQL	Cases, documents, hashes, ownership and statuses	Durable business data
MinIO/local	Actual PDF/image bytes	Configured document retention
Redis	Task messages, results and rate counters	Short-lived/queue data
Celery	No durable data itself; executes work	Worker process lifetime

Worker flow




```
docker exec grd-redis redis-cli ping
docker exec grd-redis redis-cli --scan --pattern "grd:rate-limit:*"
docker compose logs -f celery redis
```

10. Repeated requests and changing client ports

I-11: repeated GET /api/v1/cases

Observed symptom	Uvicorn logged GET /api/v1/cases repeatedly, each line showing a different client port.
Root cause	The Cases page polls every 3 seconds for updated Celery status. The operating system selects an ephemeral source port for browser connections; this is separate from the API listening port.
Resolution	No server-port change is required. Keep API target fixed at localhost:8003. Adjust or replace polling only if load requires WebSocket/SSE or a longer interval.
Validation	Every request still reaches 127.0.0.1:8003 and receives a normal 200 response.

Log part	Meaning
127.0.0.1:52473	Browser/client address plus temporary source port.
POST /api/v1/documents	HTTP method and fixed route.
HTTP/1.1 202 Accepted	Server response sent through the same connection.
Server 127.0.0.1:8003	Fixed Uvicorn listening address and port; often omitted from the log line.

Example network scenario

The browser opens a TCP connection from 127.0.0.1:52473 to 127.0.0.1:8003. The API sends 202 back through that connection. A later poll may use 127.0.0.1:52482, but its destination is still 127.0.0.1:8003. The temporary port lets Windows match response packets to the correct browser socket.

Frontend proxy

```
Vite browser URL: http://localhost:5176
Frontend request: /api/v1/cases
Vite proxy target: http://localhost:8003
FastAPI route: /api/v1/cases
```

Repeated successful polling is expected. Repeated 401, 429, 500 or 503 responses require investigation.

11. Frontend directory and dashboard preservation

I-12 and I-13

Observed symptom	Verification UI changes appeared in the wrong project or replaced the intended dashboard layout.
Root cause	The machine contained multiple dashboard/front-end directories, and the desired integration boundary was the More menu inside grd-document-verf/frontend/src.
Resolution	Restrict edits to the specified frontend source, preserve the base dashboard and expose verification features under More / Verification Command Centre and its case/upload pages.
Validation	npm run build passes and existing dashboard sections remain available.

Working directory checks

```
cd D:\newdata\grd_document_verf\grd-document-verf\frontend
Get-Location
Get-ChildItem .\src
npm run build
npm run dev
```

Frontend behavior now covered

Feature	Behavior
Submit document	Calls POST /api/v1/documents and shows API detail on failure.
Cases	Polls API, expands documents and displays observations.
Retry	Shown only for eligible FAILED, non-infected, non-expired documents.
Storage	Displays Available or Expired.
Statuses	Shows UPLOADED, QUARANTINED, SCANNING, PROCESSING and final states.
Navigation	Verification features remain under the More experience instead of removing the base dashboard.

Example UI scenario

A verification executive uploads a bank statement. The page returns immediately, then Cases shows QUARANTINED, SCANNING and PROCESSING as the worker commits each stage. If the PDF is password protected under review policy, the final badge is MANUAL REVIEW and the observation explains why.

12. Docker Compose, ClamAV and MinIO

I-14 and I-24: multi-service startup

Observed symptom	A separate ClamAV container was considered, but antivirus.py executed a local clamscan command. Services also required several manual terminals.
Root cause	A ClamAV daemon container would require a network protocol client. The existing implementation was command-based and expected the executable beside the Celery process.
Resolution	Install ClamAV and freshclam in the Celery worker image. Compose starts MySQL, Redis, MinIO, backend, Celery and Celery Beat with dependencies and health checks.
Validation	docker compose config passes; worker lists verify_document and expire_quarantined_documents tasks.

I-17: what MINIO_ENDPOINT means

MINIO_ENDPOINT is the base URL used by boto3 for the S3-compatible API. Inside Compose it is **http://minio:9000** because service names resolve on the private network. From Windows it is **http://localhost:9000**. The web console uses port 9001.

Mode	Storage setting	Endpoint
Local F5	STORAGE_BACKEND=local	Protected storage/quarantine folder
Compose	STORAGE_BACKEND=minio	http://minio:9000
Host tools	MinIO S3/console	http://localhost:9000 / http://localhost:9001
Production	STORAGE_BACKEND=minio	TLS endpoint for managed/redundant MinIO

```
cd D:\newdata\grd_document_verf\grd-document-verf
docker compose up -d --build
docker compose ps
docker compose logs -f backend celery celery-beat minio
docker compose down
```

Do not use docker compose down -v unless volume removal is intentional. The bundled MinIO is suitable for local integration; production requires TLS, changed credentials, redundancy and backups.

13. Rate limiting and retry processing

I-15: rate limiting

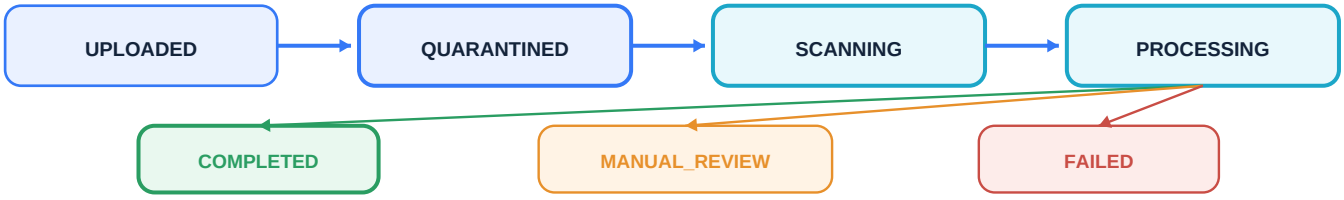
Observed symptom	Upload and API endpoints could be called repeatedly without an application limit.
Root cause	Authentication alone verifies identity but does not control request volume or expensive file-processing demand.
Resolution	Redis Lua counters now enforce general API, upload and login limits. API/upload limits use user and tenant identities; login uses client IP. Fail-closed mode returns 503 if Redis is unavailable.
Validation	Exceeded limits return 429 plus Retry-After and X-RateLimit headers.

Default policy

Scope	User/IP	Tenant	Window
General API	120	1000	60 seconds
Upload	10	100	600 seconds
Login	10 per IP	Not applicable	300 seconds

I-16: retry processing

POST /api/v1/documents/{document_id}/retry is permitted only for the owning tenant and FAILED documents. INFECTED and expired documents cannot retry. A database row lock prevents concurrent retries. If queue dispatch fails, the previous document and case state is restored.



Example retry scenario

ClamAV was temporarily unavailable, so fail-closed processing marked the document FAILED with malware status ERROR. After ClamAV is restored, the user clicks Retry processing. The API verifies the quarantine object, resets QUARANTINED/QUEUED and publishes a new task. An INFECTED result would not expose the retry action.

```
curl.exe -X POST http://127.0.0.1:8003/api/v1/documents/<DOCUMENT_UUID>/retry `
-H "Authorization: Bearer <ACCESS_TOKEN>"
```

14. Expiry and clear lifecycle statuses

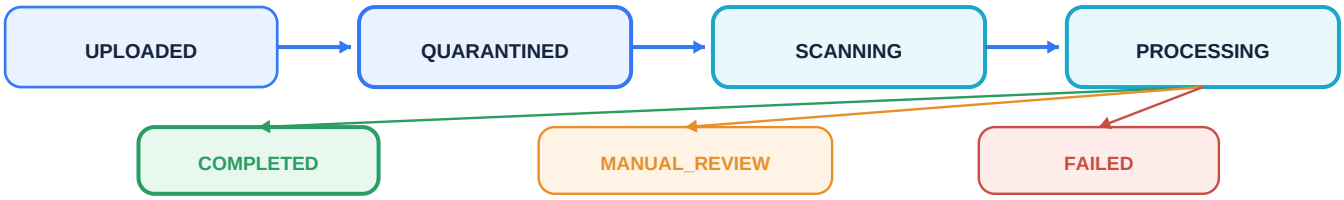
I-18: document expiry

Observed symptom	Quarantined file bytes could remain indefinitely after processing.
Root cause	No scheduler selected documents that exceeded a configurable retention period.
Resolution	Celery Beat publishes expire_quarantined_documents hourly. The worker deletes local/MinIO bytes, records storage_deleted_at and retains database metadata for audit.
Validation	The dashboard shows Storage: Expired, retry is suppressed and tests confirm content deletion plus metadata retention.

```
DOCUMENT_EXPIRY_ENABLED=true
DOCUMENT_RETENTION_DAYS=30
DOCUMENT_EXPIRY_SWEEP_SECONDS=3600
DOCUMENT_EXPIRY_BATCH_SIZE=100
```

I-19: processing statuses

Earlier states exposed internal OCR/extraction/check names and the worker jumped from quarantine directly to a final state. The public lifecycle now records clear operational stages. Legacy enum values remain readable for existing rows.



Stage	Trigger
UPLOADED	Intake request accepted in the application workflow.
QUARANTINED	Validated bytes stored with random key and SHA-256 metadata.
SCANNING	Celery starts malware scanning.
PROCESSING	Malware gate passed or policy allowed continuation.
COMPLETED	Automated processing succeeded.
MANUAL_REVIEW	Password/policy or review condition requires a person.
FAILED	Malware, storage, scanner or processing failure stopped safely.

Existing MySQL deployments receive an additive startup compatibility update for the new enum values, storage_deleted_at column and index.

15. Git history and Firefox configuration

I-20: fatal bad revision refs/heads/main

Observed symptom	VS Code Git History requested refs/heads/main and Git reported bad revision.
Root cause	The repository was initialized but had no first commit, so the main branch did not yet reference a commit object.
Resolution	Review changes, create the initial commit and explicitly name the branch main.
Validation	git log --oneline returns the new commit and VS Code history loads normally.

```
cd D:\newdata\grd_document_verf
git status
git add .
git commit -m "Initial GRD document verification implementation"
git branch -M main
git log --oneline --decorate -5
```

The .gitignore excludes virtual environments, Python caches, frontend build artifacts, active .env secrets, local quarantine data, logs and editor caches. Confirm staged files before committing.

I-21: open VS Code authentication links in Firefox

The workspace sets **workbench.externalBrowser** to **firefox**. VS Code then opens external links, including authentication URLs, with Firefox when supported by the command/extension.

```
# .vscode/settings.json
{
"python.defaultInterpreterPath": "${workspaceFolder}/backend/.venv/Scripts/python.exe",
"python.terminal.activateEnvironment": true,
"workbench.externalBrowser": "firefox"
}
```

16. Final architecture and command runbook

The resolved architecture gives each component one clear responsibility and provides safe failure behavior at every boundary.

Browser -> FastAPI -> MySQL / Redis / MinIO -> Celery / ClamAV. MySQL is authoritative for business state; MinIO is authoritative for file bytes; Redis coordinates short-lived work and limits.

Question	Final answer
Where is the file?	MinIO in Compose/production; protected local quarantine in local F5 mode.
What is in Redis?	Celery messages/results and rate-limit counters, never document bytes.
What is durable?	MySQL metadata plus MySQL/MinIO named volumes and external backups.
Who scans malware?	ClamAV executable inside the Celery worker image.
Who cleans old files?	Celery Beat schedules; Celery worker deletes through the storage adapter.
How does the UI update?	It polls tenant-scoped case data and displays MySQL status/observations.

One-command stack

```
cd D:\newdata\grd_document_verf\grd-document-verf
docker compose up -d --build
docker compose ps
curl.exe http://127.0.0.1:8003/health
docker compose logs -f backend celery celery-beat minio
```

Verification commands used during implementation

```
cd D:\newdata\grd_document_verf\grd-document-verf\backend
python -m pytest -q
python -m ruff check app tests --ignore B008
python -m compileall -q app
cd ..\frontend
npm run build
cd ..
docker compose config --quiet
```

Recorded result: 27 backend tests passed, Python lint/compilation passed, the frontend production build passed, both Celery tasks registered and Docker Compose configuration validated. Container runtime launch remains a local operational step whenever Docker Desktop is stopped.